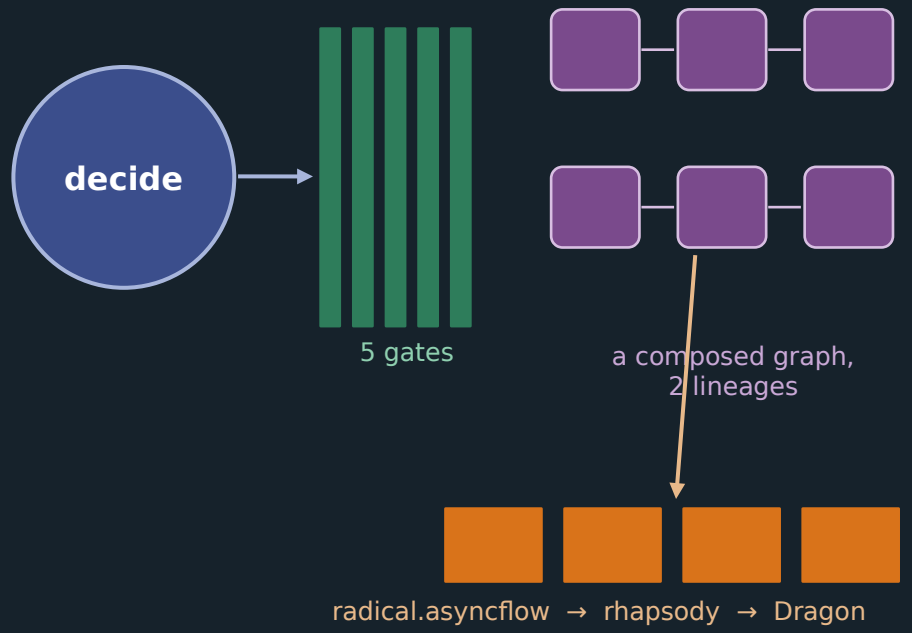
IMPRESS-A

Composing the pipeline instead of writing it down

— what that buys over an adaptive pipeline that already works, what it costs, and which of the three I cannot yet show you

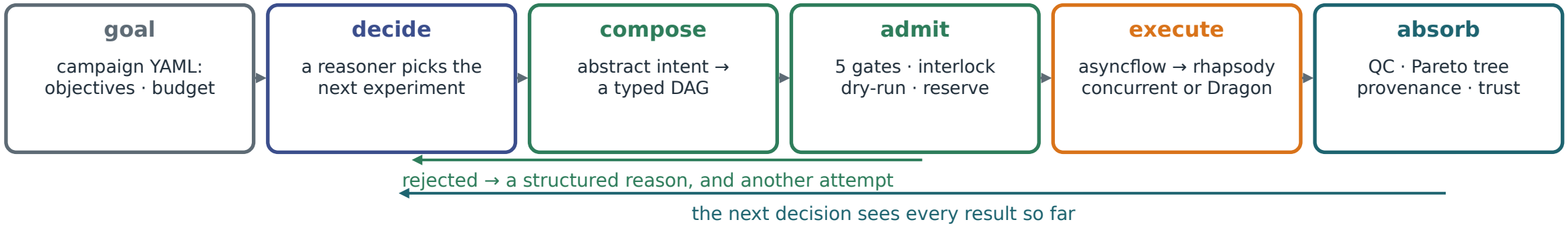
Job 22692304: six stages, 6/6 tasks ok in 2m53s, admitted on the first attempt, all four objectives valued. Job 22728140: the same shape promoted, then 3 consecutive trusted cycles with no demotion



Lab code walk, in three acts · agent/ · merge-impress-a @ c163fe2 · 6.6k lines of source, 168 tests, 11 tools in 5 toolkits

WHAT IT DOES

One goal, a campaign of composed experiments



What came back, for real — job 22692304

- rfd3_design → ligandmpnn_design → packmin → fastrelax → filter_shape → boltz_predict, 6/6 tasks ok in 2m53s
- all four campaign objectives valued: total_score -272, shape_complementarity 0.591, complex_plddt 0.507, ligand_iptm 0.752
- admitted on the first attempt, 0.27 GPU-h and 0.06 CPU-h charged against the ledger
- per-tool cost measured for all six, and the specs corrected from it

And what one completed campaign is not

- Every run so far is ONE lineage, on ONE of four GPUs. The reference pipeline's comparable runs carry 16 to 256 concurrent pipelines, so this is not a scale result and nothing here is a throughput comparison.
- No node has reached plain pass. This one is suspect because its pattern was still provisional; all 5 trusted nodes since missed an acceptance threshold instead.
- No measurement has ever superseded a prediction, so the calibration machinery — the one thing a composed graph gets that a written-down one does not — is untested against reality.

Read it as a floor, not a result. The engineering claim is that a machine-composed workflow survived contact with real tools on real hardware and was caught, costed and recorded on the way through. The science is one design.

Numbers from slides/run.json. Delta figures transcribed from plans/first-real-run.md; the raw logs live under $WORK_DIR on Delta.

WHAT WE ARE COMPARED AGAINST

The baseline is not a blank page

IMPRESS, the reference adaptive pipeline — what it already does

A40, 1-8 nodes, 8-256 pipelines

24,026	913	267.9	71%	0.053
task directories, 4 campaigns	passing folds, 0 task failures	GPU-hours billed	GPU busy at 8 pipelines per GPU (12% at one)	GPU-h per passing fold (from 0.159)

Its orchestration is not the overhead either: the adaptive callback is 0%–2% of the manager's wall clock and backend dispatch has a median of 0.001s, and its pipelines are busy 96%–98% of the time.

And what it costs to change it: one use case is a 1,450-line pipeline class plus a 574-line runner; its non-adaptive variant is 102 lines.

where the pipeline's shape lives	what is actually written down	file
the stage graph	19 next_step assignments in one 220-line decision function, over 7 STEP_ constants	`run_small_molecule_binding.py:210`
the thresholds	a PROD dataclass in the runner — fa_rep < 100, total_score < -250, interaction_energy < -8	`…:63`
the QC verdict	computed inside the coroutine that parses the output, then read back out of pipeline.state	`small_molecule_binding.py:1154`
the adaptive hook	a plain Python closure with full object access, defined in the runner script	`briefs/impress.md:112`

Its adaptivity is a re-sampler, not a ratchet

- A guided child against its OWN parent fold: ΔpLDDT +0.011, better in 51% of 602 pairs, p = 0.85. Depth plateaus by the middle third (90.7 → 95.2 → 95). Still worth having: 8.1 tasks per fold against 34.8 from scratch.

And its yield needs a novelty haircut

- 804 passing folds are 596 clusters at ≥90% identity — 1.3 folds each. Between different pipelines, median identity 0.11. Width buys independent searches; depth refines one. That is the argument for lineages and a front, and act 2 says what we have run.

Said plainly, because it is owed: it paid for lessons we inherited for free, including the NVMe fix this project ported out of its PR #67. Nothing here divides its numbers by ours.

source: IMPRESS small-molecule performance report (impress-smallmol-performance/report.md), over impress_runs_export_nvme_2026-10-08.tar.gz; structural counts mined from examples/small_molecule_binding in this repo at c163fe2.

HOW TO READ THIS DECK

Three dimensions, and the one I have least evidence for

FUNCTIONALITY

What specific tasks can you ask for that an adaptive pipeline cannot be asked? Not features — tasks, carried end to end with no human in the middle.

what it answers

Composed per cycle, not written down

five gates and a dry-run before anything executes

11 tools, zero composer edits

a tool is a spec.yaml, a task agent and mandatory gates

Four control models, one executor

byte-identical below the policy layer, which is what makes them comparable

PERFORMANCE

What does composing, validating and scoring a graph cost — in wall clock, in hardware, and in scientific yield? Those three are not linear in each other.

what it answers · and what it does not

Overhead 2.8s of a 159s run

and scrutiny itself costs 0.0s

GPU ≤17%, tasks 95% of elapsed

one GPU works, three idle — we have never packed one

17 designs in 5 seed groups

and no node has ever reached a plain pass

USABILITY

Not the interface — the use cases. Is this an application someone opens, a platform other people plug into, or a component something else drives?

what it answers

As an application — yes

one YAML, one command, 168 tests with no allocation

As a platform — yes

runtime-discovered toolkits, and a control plane in two transports

As a component — no evidence

nothing else has ever driven it, and there is no resume

The weak column is the middle one, and specifically performance at scale. Every figure in it comes from one lineage on one GPU: 16 complete six-stage runs, all of them serial. On real tools replicas > 1 has never executed — only against mocks — so the invariant that N lineages must produce N independent nodes is unexercised, and it is the one most likely to be silently wrong. There is no head-to-head against the reference pipeline and I am not going to construct one.

Each dimension gets an act, in this order, and each act names its own weakest claim before it rather than after.

WHY IT LOOKS LIKE THIS

Two facts set the whole design

A machine composes workflows nobody has reviewed — out of scientific tools that fail **silently** far more often than they crash. A generator returns a structure with no secondary structure, a confident designability score, and exit code 0.

consequence	what it means in the code	where
A policy emits intent, never a DAG	ExperimentIntent is a list of tool ids and parameters. Turning it into a graph is the composer's job, so control models stay interchangeable — and the import contract forbids the edge.	`policy/base.py` · `core/decision.py:21`
Composition-time validation, then a dry-run	Five gates, then every task agent parameterizes without executing. Rejection is cheap and structured, so free composition is affordable to get wrong.	`compose/validate.py:30, :143`
Trust is scored on shape, and earned	A novel graph shape runs under a cost cap, a forced dry-run, and is marked suspect whatever it scores. Promotion is N consecutive integrity-clean runs; demotion is immediate.	`compose/interlock.py:146`
Exit code is never evidence	Every non-P6 tool declares mandatory QC gates, and a node whose verdict is FAIL is never eligible for the Pareto front, whatever it scored.	`tools/agent.py:110` · `core/pareto.py:56`
One writer, and it decides termination	The executor owns all campaign state; every mutating block is await-free, so an observation is never taken mid-absorb. Budget and stagnation are facts a reasoner cannot see.	`runtime/executor.py:242`

The flip side, said plainly: if a human reviewed every graph before it ran, you would not build most of this — you would write the DAG down once and schedule it. That is the right answer under review, and next door it is 1,450 lines of pipeline class per use case. Everything in act 1 is what you get for not writing it down; everything in act 2 is what that costs.

ACT 1 OF 3

FUNCTIONALITY

What specific tasks can you ask for that an adaptive pipeline cannot be asked?

You state a goal, not a pipeline

An ExperimentIntent is a list of tool ids; the composer types it, five gates check it, and a dry-run parameterizes every stage before anything executes. The reference pipeline's shape lives in 19 next_step assignments inside one decision function.

A tool is a spec file, and QC is part of the spec

11 tools in 5 toolkits, each a spec.yaml plus an agent. Adding one never edits the composer, the validator or the manager - and gates are mandatory, shared by id, and not written by whoever parses the output.

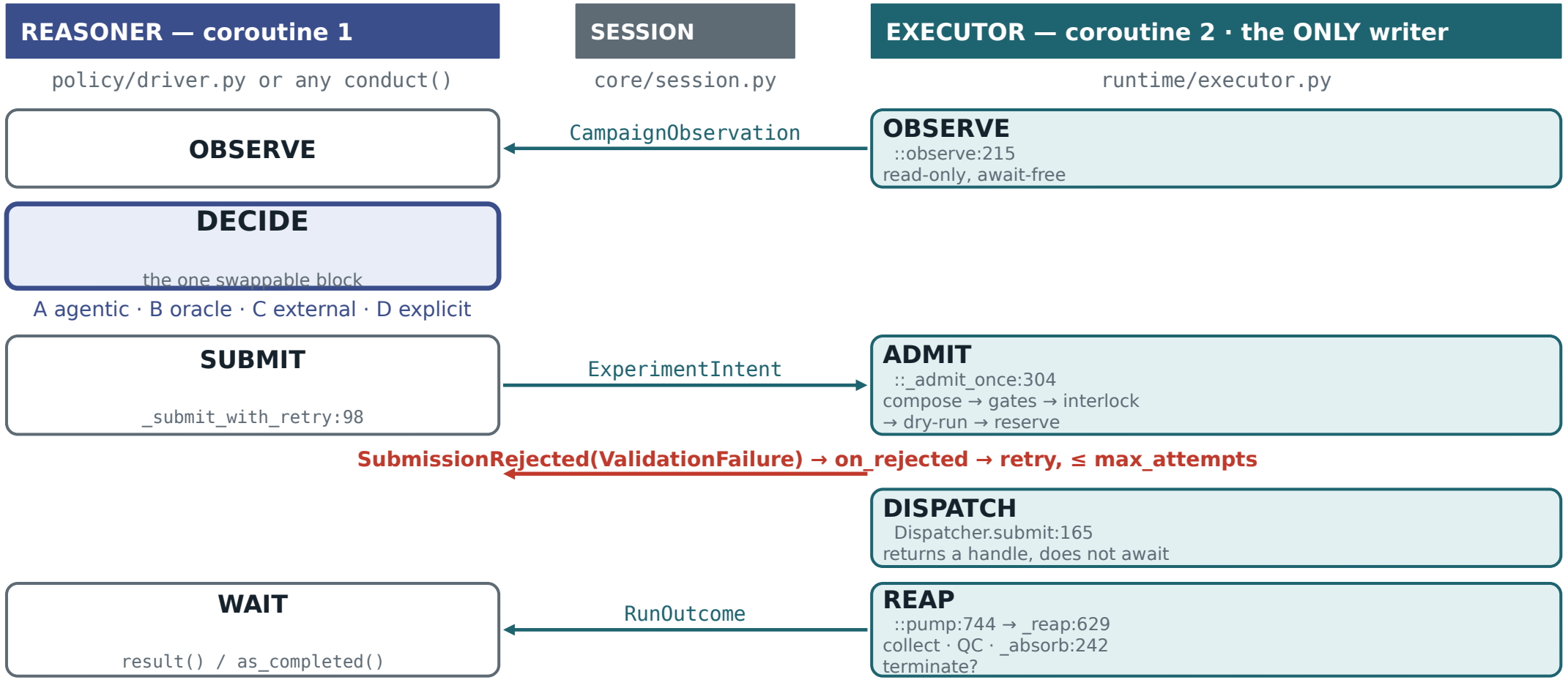
The control model is swappable, and the layer below it does not know which one ran

Four models plus conduct() reasoners over one executor, byte-identical below the policy layer - which is what makes comparing an LLM against a rule cascade mean anything at all.

The weakest claim in this act: every graph we have ever composed is the same six-stage chain the reference pipeline already had. Free composition is real in the code and unexercised in the evidence.

THE LOOP · F1

Two coroutines, one writer



Why the split exists

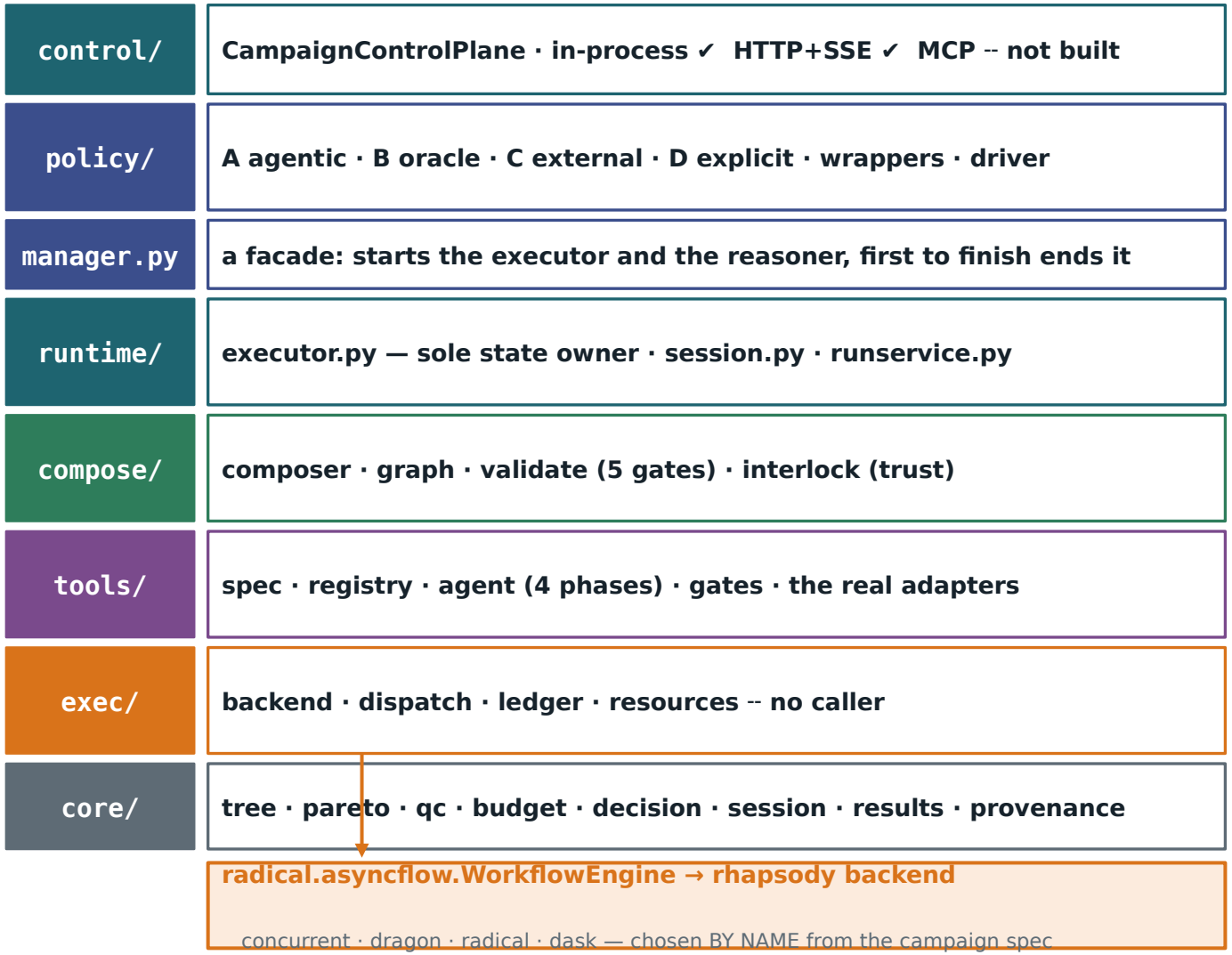
- The old manager awaited the whole graph where it submitted it, so the policy could not be consulted until the slowest lineage finished.

What keeps it honest

- Every mutating block in the executor is await-free. Adding an await inside observe or _absorb reintroduces torn reads — a discipline, not something Python enforces.

ARCHITECTURE · F2

Layers, and the four arrows that are not there



The two rules that carry weight

- policy ↛ tools | exec | runtime — a policy that can reach a tool adapter stops emitting abstract intent, and control models stop being swappable.
- compose ↛ exec — validation stays testable with no backend at all. That is the entire laptop test tier.

Asserted, not reviewed

- tests/test_layering.py walks the source with ast and checks every internal import against one table.
- ast.walk deliberately: adapters defer science imports into run() bodies, and a deferred cross-layer import is exactly as fatal as one at the top.

What writing it found

- Two undocumented edges — manager → tools, and cli / __main__ in no row at all. Both are recorded now: a contract with undocumented edges is not a contract.

policy → core is the edge that forced core/session.py and core/results.py to exist: a reasoner may hold a RunOutcome, so RunOutcome cannot live in exec/.

ADMISSION

Admission: five gates, an interlock, and a reservation

gate	rejects	line
1 type	an edge whose producer type does not unify with the consumer's	:38
2 structure	cycles, empty graphs, a tool composed without its QC gates	:70
3 parameter	out of range, unknown, or declared frozen	:86
4 resource	a SCHEDULED P6, oversize shapes, unproven GPU API, external submission	:103
5 budget	an estimate exceeding remaining budget in any dimension	:134

then	what it does	where
INTERLOCK	one instance in flight for an untrusted shape; a cost cap for a provisional one — both transient	executor.py:331, :337
DRY-RUN	every task agent parameterizes WITHOUT executing	validate.py:143
RESERVE	re-check against AVAILABLE budget — remaining less what in-flight runs hold	executor.py:355

Why RESERVE is last: the dry-run awaits. By the time it returns, another submission may have claimed the budget gate 5 saw, so reserve re-checks and is the authoritative decision.

A pattern is the graph's SHAPE

- tool ids plus typed edges, hashed. Parameter values are excluded, so tuning a parameter does not reset accumulated trust.

	trusted	provisional
cost ceiling	normal	10% of available
dry-run	skipped	forced
result	its QC verdict	SUSPECT anyway
in flight	many	one

Promotion is mechanical

- N consecutive integrity-clean runs promote; any integrity failure demotes and resets the counter.
- An untrusted shape may have only ONE instance in flight — N concurrent copies are one draw sampled N times, not N pieces of evidence.

Be honest about the limit: the interlock buys examination and delay, not soundness. A *consistent* novel silent failure promotes.

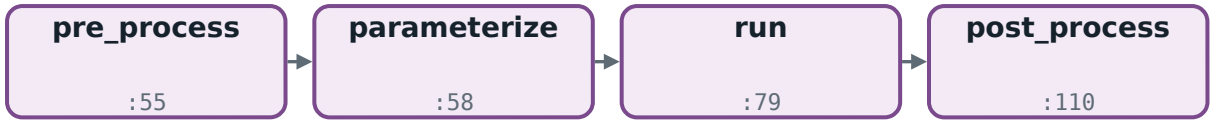
All of it is one method — runtime/executor.py::_admit_once:304 — and it dispatches nothing. Admission is synchronous everywhere, including over HTTP.

TOOLS

A tool is a spec file, and QC is part of the spec

```
id: filter_shape
pattern: P2                  # compute pattern — normative, see docs
entry: impress_a.tools.rosetta_agents.FilterShapeAgent
resources: {cores: 2, walltime_s: 120}
inputs:   {structure: {type: Complex}}
outputs:  {structure: {type: Complex}}
metrics:  [shape_complementarity]
qc_gates:
  - {id: output_present,  params: {key: result}}
  - {id: metric_in_range, params: {metric: shape_complementarity,
                                   min: 0.55, max: 1.0}}
cost_model: {unit: invocation, cost: {cpu_hours: 0.01}}
```

toolkits/rosetta/tools/filter_shape/spec.yaml · EDITED



post_process ENFORCES the spec's qc_gates — never skippable, and execute() is not overridable by an adapter. Loading is validating too: a default outside its own range, an unknown gate id or a SKILL.md missing a section is a load-time error, and a toolkit that fails anywhere registers NOTHING rather than a partial set.

	here	the reference pipeline
the verdict	declared in the spec, resolved by id, mandatory — and a FAIL node never reaches the front	computed in the coroutine that parses the output, as 'pass' in pipeline.state
the thresholds	in the spec, beside the tool	in a PROD dataclass in the runner (14 fields)
who is stricter, on the two stages we share	total_score ≤ 0, fa_rep ≤ 500, no interaction_energy	**< -250, < 100, < -8 — THEY ARE**

The suite manufactures the failure mode

```
class NoodleAgent(TaskAgent):
    """A tool that LIES.

    Completes successfully, returns confident-looking numbers,
    and produces a structure with no secondary structure.
    """
    async def run(self, req, params):
        return {"result": "backbone", ...,
                "outputs": {"designs": "noodle"},
                "metrics": {"ss_fraction": 0.02, "designability": 0.91}}
```

tools/mock_agents.py:84–95 · EDITED

```
assert not res.failures, "the tool must SUCCEED - that is
                          what makes it silent"
assert res.merged_metrics()["designability"] > 0.9
assert res.merged_qc().verdict is QCVerdict.FAIL
```

tests/test_campaign.py:96–98 · EDITED (wrapped, messages dropped)

Where this defence is still thin

- mock_noodle exists so the QC layer is tested at campaign scale against a tool that lies. The real toolkits have no equivalent.
- Their gates are almost entirely metric_in_range against a number the tool chose to report about itself — which is exactly what a confidently-wrong tool passes.
- No structural gates: nothing checks the ligand is in the output complex, that there are no chain breaks, or that sequence length matches the contig. The reference pipeline gates on RFD3's clash counts; we open that sidecar JSON and do not read it (backlog B1/G2).

Our thresholds are mined from the spec files, theirs from their runner's PROD dataclass — the one comparison in this deck where we are behind.

ACT 2 OF 3

PERFORMANCE

What does composing, validating and scoring a graph cost - in wall clock, in hardware, and in scientific yield?

The loop is not the cost

Over 16 six-stage runs, everything outside a task - dispatch, absorb, QC, provenance - is a median 2.8s of a 159s run, and running untrusted costs 0.0s [-0.9, 1.2]. The reference manager's adaptive callback is the same order: 0%-2% of wall.

The failures that eat allocations are bounded, in code

Backend startup and teardown both, measured against two lost allocations and the 64 GPU-h a teardown that never returned cost the reference pipeline - 37% of that job's billed total, spent after the science was done.

The headroom is not orchestration

Tasks fill 95% of elapsed, so the idle hardware is the three GPUs we never give work to - and packing is the lever the reference pipeline already pulled, from 12% to 71%.

The weakest claim in this act: there is no serial baseline and no head-to-head. One lineage, one GPU, three idle, on a different partition from every reference figure - so nothing here divides into a speedup.

★ One generic factory, and a submit that does not await

```python
def _make_task(self, node_id, tool_id, params, invocation="", seed=None):
    """ONE generic factory for every node in every graph."""
    reg, spec = self.reg, self.reg.get(tool_id)
    agent_cls = reg.agent_for(tool_id)
    workdir = self.workdir      # bind locally: the closure is PICKLED

    async def _run(*deps):
        agent = agent_cls(spec)
        req = TaskRequest(tool=tool_id, params=params, seed=seed, ...)
        res = await agent(req)
        return {"tool": ..., "metrics": ..., "qc": ..., "cost": ...}

    _run.__name__ = node_id   # asyncflow reads the name from __name__
    return self.flow.function_task(_run)
```

exec/dispatch.py:144–163 · EDITED — annotations elided

```python
for tid in g.topo_order():                # edges via unawaited futures
    deps = [futures[d] for d in g.nodes[tid].deps]
    futures[tid] = tasks[tid](*deps, workflow_id=run_id)

# one bad task must not cancel its siblings
gather = asyncio.gather(*futures.values(), return_exceptions=True)
return DispatchHandle(run_id=run_id, graph_id=g.id,
                      futures=futures, gather=gather)   # NOT awaited
```

exec/dispatch.py:182–190 · EDITED — see CODE_FOR_DECK.md

Two asyncflow idioms, not interchangeable

- An unawaited future passed as an argument expresses a DAG edge.
- asyncio.gather groups INDEPENDENT work — the replica lineages. The executor, not the dispatcher, decides when to wait on it, racing every in-flight gather with FIRST_COMPLETED.

The bet, and it held twice

- A DAG described as data dispatches with one generic factory — no codegen, no AST work, no templating.
- And the same factory survived going non-blocking unchanged: submit builds the gather and hands it back instead of awaiting it.

Three findings in four lines

- The task decorators take no name= kwarg — setting __name__ first is what lets one factory serve every node.
- workflow_id tags every task, but no lookup API exists, so the handle is the run's own index.

Cancellation is advisory

- Measured, not assumed: Future.cancel() returns False once a callable has started, and asyncflow discards that answer. Queued work is reclaimed, running work is not, and you cannot learn which happened.
- So a run's terminal state always comes from collecting it. Backtracking sidesteps this by branching the tree; reclaiming a GPU from an abandoned run is unsolved.

★ Building the engine on the loop that will use it

```python
async def make_engine_bounded(kind, config, timeout_s, heartbeat_s, work_dir):
    if timeout_s <= 0 or kind.lower() in _CHEAP_KINDS:
        return await make_engine(kind, config)

    # ONLY the synchronous construction leaves this loop.
    threading.Thread(target=_construct_backend_in_thread,
                     args=(kind, config, fut), daemon=True).start()
    be = await asyncio.wait_for(asyncio.shield(async_fut), timeout_s)

    async def _finish():                    # on OUR loop, deliberately
        backend = await _init_backend(be)  # __await__ registers states
        return await WorkflowEngine.create(backend=backend, work_dir=work_dir), backend
```

exec/backend.py:128–176 · EDITED — condensed, see CODE_FOR_DECK.md

Backends are awaitable, and must be awaited __await__ triggers _async_init(), which registers task states. Constructing synchronously appears to work, then fails later with "Backend 'x' not registered. Available backends: []".

The engine must be built on the loop that will submit to it WorkflowEngine.__init__ captures the running loop and puts its run-component dispatch task on it. Build one on a throwaway loop and you get an engine that looks fine and dispatches nothing — silently.

Real backends live in rhapsody, not asyncflow asyncflow exports only Noop and Local. Everything real resolves by NAME through rhapsody.backends.get_backend(), from the campaign spec — so no backend class is named outside one module.

Dragon empties the root logger Pool()/ProcessGroup() call setup_BE_logging, which clears root handlers. A run configured with logging.basicConfig alone goes mute the moment the backend comes up.

What each lesson cost

22328172 22328262 **two full allocations**	the engine was built on a throwaway loop via asyncio.run on a helper thread — every submitted task hung
22318678 **one 2-hour allocation, in full**	Dragon's Batch() builds synchronously with no await points. No heartbeat, no campaign log, nothing, until the wall clock killed it
22491438 **~64 GPU-hours, 37% of the bill**	flow.shutdown() never returned after the science finished. Teardown is the more expensive end

Both ends are bounded now

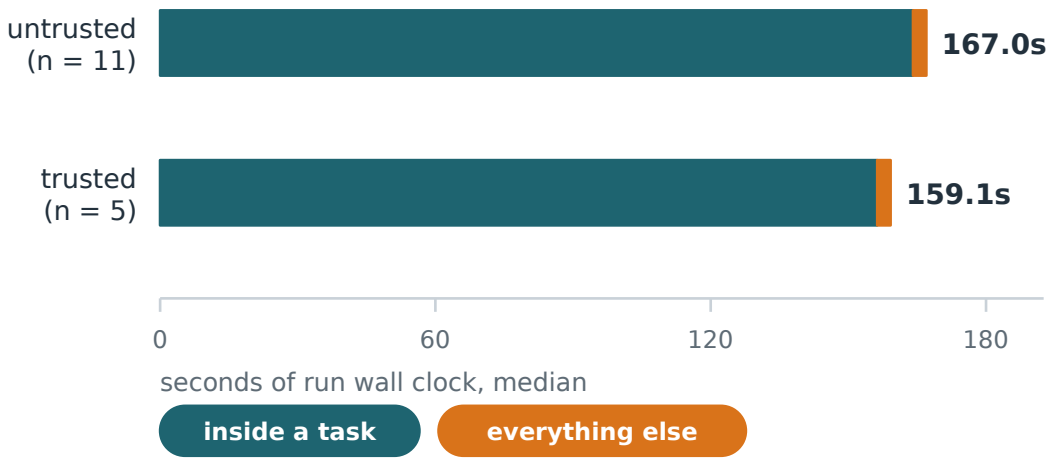
- backend_startup_timeout_s / _heartbeat_s and backend_shutdown_timeout_s. A stuck teardown is abandoned with a warning, never raised — results are already durable by then.

The reference IMPRESS pipeline declined the teardown fix and relies on an operator watching the log. We do not.

PERFORMANCE · MEASURED

What the loop costs, and what is not measured

A six-stage run, by where its seconds go



Everything else is dispatch, absorb, QC, Pareto, provenance and the ledger: a median 2.8s. Running UNTRUSTED — forced dry-run, cost cap, suspect verdict — costs 0.0s [-0.9, 1.2] on top of that. The gates are not the cost.

Declared vs measured, per six-stage chain: 0.17 GPU-h declared, 0.0285 measured (max 0.0337). Every declared figure is ≥2.25× the worst task observed, deliberately — gate 5 and the untrusted cap REFUSE graphs against them, so they err high.

Measured, and in this deck

- 101 timed tasks over 5 jobs, from asyncflow's own RUNNING → DONE lines in campaign.log — the only per-task wall time a run records.
- Per-stage spread: rfd3 41s median, 15 of 17 tasks in 40–45s, one outlier at 145.7s.
- Where the allocation goes: tasks 95% of elapsed, CPU efficiency 42%, peak memory 16.3 of 240 GB.
- Queue wait, unbilled but real: 184–26,626s against campaign elapsed of 191–821s. It is the real latency of a test cycle, and it is nobody's code.

Not measured — so not claimed

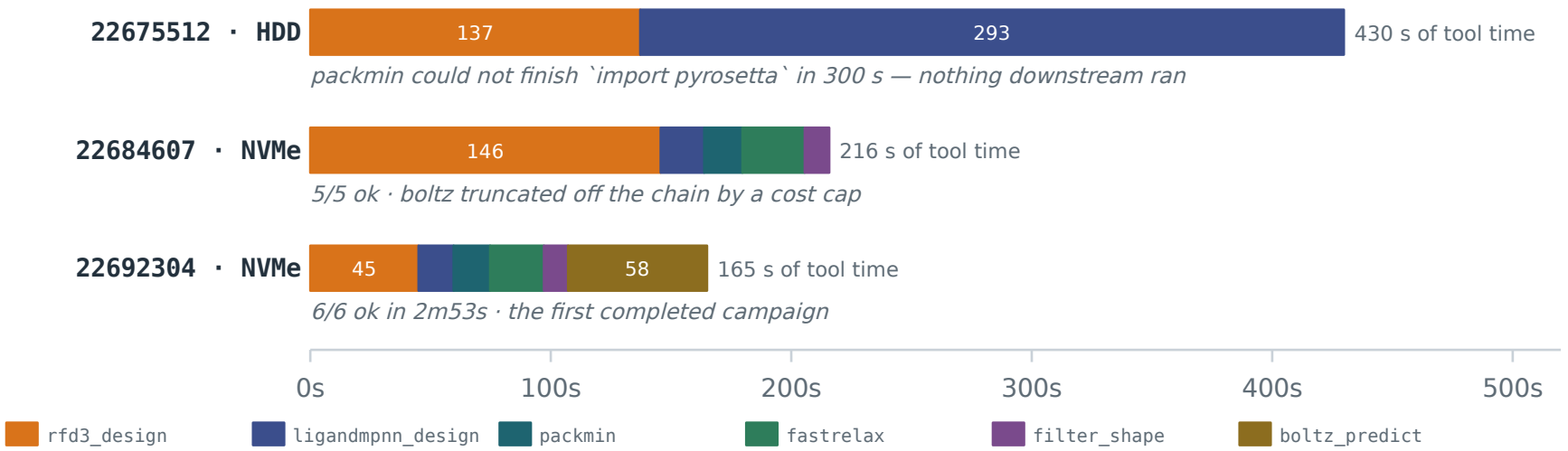
- No serial baseline exists, so nothing on this slide divides into a speedup.
- GPU utilisation is recorded nowhere in a run. ≤17% of billed GPU time is an UPPER BOUND with model load counted inside it.
- Concurrency has never happened: every run is one serial chain, so this is not a scheduling result. Packing more than one lineage onto a GPU has never been attempted.
- The ledger's cost field is the admission estimate echoed back. It is not evidence and no figure here uses it.

Where this stops being a systems question: the loop is ~2% of a run and the hardware is 83% idle. The next real gain is not in this orchestrator — it is giving the other three GPUs work, which is a lineage-count question, not a scheduling one.

source: performance_analysis report 2026-10-09 over impress_a_runs_2026-10-08.tar.gz (22702568, 22726105, 22728140 trust; 22684607, 22692304 smoke), code at d3c20ca

MEASURED · F3

Seven jobs: what storage bought, and what trust cost



Then three jobs about the SHAPE, not the science — 15 runs, 6/6 tasks every time

run	22726105 · 13m41s	22728140 · 13m37s
r0001	provisional · clean	provisional · clean
r0002	provisional · clean → PROMOTED	provisional · clean → PROMOTED
r0003	**trusted · fail (acceptance)**	**trusted · fail (acceptance)**
r0004	**trusted · INTEGRITY fail → demoted**	**trusted · fail (acceptance)**
r0005	provisional · fail (acceptance)	**trusted · fail (acceptance)**

22702568 ran 30/30 tasks clean and promoted nothing — the all-gates rule made promotion a function of target difficulty (decision 0013). Two defects, both fixed: packmin's total_score <= 1000 bound was ONE observation wide and demoted r0004 over a +1064.5 pose fastrelax took to -336, and the driver counted a Backtrack as a cycle, so every job asked for 6 runs and ran 5.

Where the bytes live

- import pyrosetta off HDD-backed Lustre: 471 s for a 598 MB rosetta.so, demand-paged. torch: ~280 s.
- On NVMe both are seconds — the single variable behind rows one and two.
- And we did not find it: the $WORK_DIR move is ported from their own PR #67.

Wall time is not stable

- rfd3_design took 145.7 s in 22684607 and 45 s in 22692304 — same campaign, same allocation shape, same parameters, 3.2× apart.

So do not tune cost models to it

- They feed gate 5 and the untrusted cap, which REFUSE graphs. They sit a few multiples above measurement on purpose.

Durations transcribed from plans/first-real-run.md; the trust runs from plans/next-run-promotion.md and plans/next-run-sustained-trust.md — the raw logs live under $WORK_DIR/impress_a_runs on Delta. Bars are tool time.

PERFORMANCE · THE SCIENTIFIC HALF

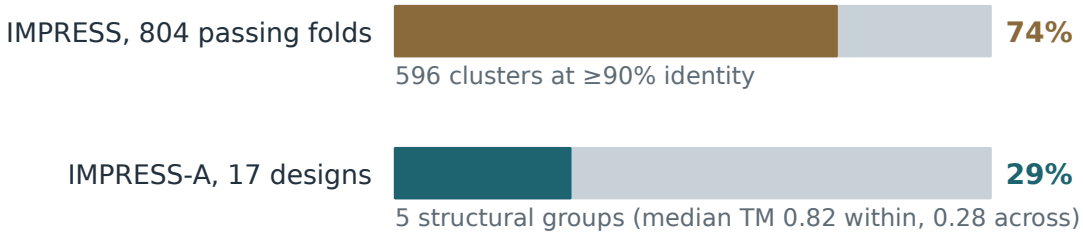
Where scale stops buying yield

Its search improves, then plateaus — by the middle third



median complex pLDDT by episode tercile, pooled over its clean runs — and median ligand_iptm does the same: 0.845 → 0.889 → 0.896

And the headline count is not the yield



independent samples as a share of the headline count, 0–100%. Both analyses overcount their own yield, and both say the same thing: width, not depth.

The honest state of this slide: the machinery for width is written, typed and unit-tested, and replicas > 1 has run against mocks only, never on the real chain. Worse, the cheap route to it is closed: the pattern signature includes replica multiplicity, so the trust earned at one lineage does not transfer to four — a four-lineage graph starts untrusted, under a cap it already exceeds. That is backlog A10 and it is ask 5.

What this project does about it

- replicas: N means N INDEPENDENT lineages, one DesignNode each — the composer fans out and the executor's _absorb never funnels them back into one candidate.

N lineages, N nodes

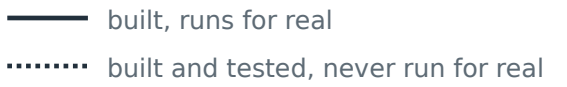
`compose/composer.py · executor._absorb`

A front that is not monotonic: a measurement supersedes but never deletes a prediction, so a node promoted on an optimistic guess can be demoted by its own assay

`core/tree.ingest_measurement · core/pareto.py`

Trust scored on graph SHAPE, so breadth has an identity of its own and cannot inherit credit earned at width 1

`compose/graph.py:51`



Our own 17 designs: 0 byte-identical backbones, median Cα RMSD 2.39 Å. The seed plumbing does something — but at one lineage per run, the seeds come from the run label, so breadth and reproducibility are the same unexercised question.

source: IMPRESS small-molecule performance report (impress-smallmol-performance/report.md), over impress_runs_export_nvme_2026-10-08.tar.gz · source: performance_analysis report 2026-10-09 over impress_a_runs_2026-10-08.tar.gz (22702568, 22726105, 22728140 trust; 22684607, 22692304 smoke), code at d3c20ca

WHAT A DRY RUN CANNOT SEE

Four defects that reached real hardware

1	**What a tool writes**	The *.cif.gz glob selected a diffusion TRAJECTORY, not the design — same extension, and `denoised` sorts first. A 5.7 MB multi-frame stack went downstream as the backbone, against a 19 KB design.	**now:** Discovery goes through the sidecar *_model_*.json now, and an out_dir it cannot identify a design in raises rather than guessing.
2	**Whether a tool can import**	LigandMPNN died in run.py's MODULE-LEVEL imports, before parsing an argument: a missing ml_collections, then np.int, removed in numpy 1.24 and still used by its vendored openfold.	**now:** It runs through MPNN_SHIM now — the shim restores the aliases and hands off via runpy — never run.py directly.
3	**Where the bytes live**	import pyrosetta cost 471 s off HDD, so packmin could not finish starting inside its 300 s budget. The queued fix was to raise three walltimes 6×, which would have hidden the real cost.	**now:** WORK_DIR on NVMe; `impress-a preflight` now flags a venv still on slow storage.
4	**An estimate that refuses work**	cost_model figures were literature guesses 6–69× over measurement. The inflated GPU side summed past the untrusted cap, the chain was refused, and the policy truncated boltz_predict off the end — silently.	**now:** Costs corrected from measurement, and _objectives_without_a_producer now WARNs. It is a heuristic, not a gate.

Two things to carry away. "Checked against the binary" is not "executed" — numbers 1 and 2 were both invisible to every contract check we had. And a plausible explanation is not a diagnosis: the trajectory bug was a convincing cause for a LigandMPNN failure it had nothing to do with.

TRUST

The ledger that forgot — and what it still forgets

1 · Promotion was structurally unreachable on Delta, and it looked exactly like "not enough runs yet"

```
# executor.py loaded the ledger from Path(spec.root) / "_trust",
# and spec.root defaults to the RELATIVE "campaigns/_runs".
# delta_gpu_run.sh cds into $WORK_DIR/impress_a_runs/$SLURM_JOB_ID:

  22684607/campaigns/_runs/_trust/cuda.jsonl    0 clean
  22692304/campaigns/_runs/_trust/cuda.jsonl    1 clean
```

fixed in 52332c6 — CampaignSpec.trust_root

…and then it remembered

- trust_root is absolute now, so evidence accumulates ACROSS jobs: 22702568's last event was clean, so 22726105 promoted a run early.
- 22728140 then ran 3 consecutive trusted cycles — 0 demotions, 0 integrity failures.

How it hid: nothing ever logged where the ledger was, so a file that silently reset looked identical to one that had not earned promotion yet — and it was reported that way. Campaign start now prints the absolute path with a pattern and trusted count. It had been true since the first Delta job, so **the trusted path did not execute until job 22726105**: every graph composed on real hardware before it ran under the forced dry-run and the 10% cap — and that cap is what truncated boltz_predict off the chain twice.

2 · And a pattern's identity includes its BREADTH — found while building this deck

```
parts = sorted(                       # ONE string per NODE
    f"{n.tool}<-{','.join(sorted(self.nodes[d].tool for d in n.deps))}"
    for n in self.nodes.values()
)
return hashlib.sha256("|".join(parts).encode()).hexdigest()[:16]
```

compose/graph.py:51 · VERBATIM + one added comment

N lineages give N duplicate parts, so the hash changes with the replica count — the way it deliberately does not change with a parameter:

the real six-stage chain	nodes	signature
replicas: 1	6	f5b21d82924fbcd0
replicas: 2	12	f8bcefed63c493ae
replicas: 4	24	78e730c67750a361
replicas: 1, num_designs: 3	6	**f5b21d82924fbcd0 ← unchanged**

What that costs, concretely

- Backlog A10's cheapest route — promote at replicas: 1, then run replicas: 4 — does not work. That is a different, untrusted pattern: still capped, refused at 0.68 vs 0.6 gpu-h, and on_rejected would silently shrink it to three lineages.

The question for the room

- Is trust earned at one lineage evidence about four, when the cap it would lift exists to bound blast radius?

Signatures computed live by slides/run_model.py against this checkout — the mock run on backup B1 shows the same shape, the shrink from 3 lineages to 2 producing a new pattern.

ACT 3 OF 3

USABILITY

Not the interface - the use cases. Is it an application, a platform, or a component in something larger?

As an application: yes, and without an allocation

A campaign is one YAML and one command. The whole local tier - 168 tests over 3,855 lines - runs on a laptop with no scheduler, no GPU and no cluster.

As a platform: yes, with a seam that is not the CLI

Toolkits are discovered at runtime, not packaged: 11 tools declared in spec.yaml with a SKILL.md beside them. The control plane ships in two transports, and a reasoner in another process has driven a campaign over HTTP + SSE.

As a component: no evidence, and that is the honest answer

Nothing else has ever driven it. There is no resume, no serve, no reason command, and sites/*.yaml is read by no code at all.

The weakest claim in this act: the component claim is empty - and because serve and reason do not exist, --model C fails open and reports a model-D campaign as model C.

As an application: one YAML, one command

```
source .venv/bin/activate          # NOT system python — see CLAUDE.md
pip install -e ".[dev]"

pytest tests -q                    # 168 tests, no allocation
impress-a tools                    # list the registered tools
impress-a preflight campaigns/<spec>.yaml      # BEFORE sbatch
impress-a run campaigns/mock-stabilize.yaml --model D
#   --model A|B|C|D|null|replay    --no-guard  drops the correction wrapper
```

file	what it covers
`test_core.py`	pure logic — tree, Pareto, budget, QC
`test_validation.py`	the five gates, the interlock, the trust ledger
`test_campaign.py`	full campaigns on a concurrent backend with mock tools
`test_real_toolkits.py`	the real specs, without executing any real binary
`test_backend_bound.py`	startup timeout/heartbeat against a fake backend that blocks its own thread's loop the way Dragon's Batch() does
`test_http_adapter.py`	a campaign driven end to end over a loopback socket
`test_remote_session.py`	a reasoner in another process, over the control plane
`test_layering.py`	the import contract, walked with ast rather than trusted to review
`test_gate_fixtures.py`	each real tool's gates against its known-bad fixtures, and the registry against the directories on disk

The whole local tier runs on a laptop

- A complete campaign — real manager, real policies, real composer and validator, stubbed science. HPC iteration is slow and expensive, so almost everything is verifiable locally.
- The corollary: everything only verifiable on HPC is unverified. That is the honest price of a tier that needs no scheduler.

Two conventions worth stealing

- Counts are not restated in prose. test_gate_fixtures derives the tool count from the directories rather than asserting a number.
- A magic number in a test is often a bug report: stagnation_limit = 10_000 appeared in three tests before anyone noticed the defect was in the executor, not the setup.

A 20-second demo, if the room wants it

- pytest tests -q -k lying — the bundled tool that succeeds, reports 0.91 designability, and is failed by QC anyway.
- impress-a run campaigns/delta-small-molecule-smoke.yaml --model D must compose rfd3_design…, never mock_… — that confusion is the bug that made the smoke campaign necessary.

USABILITY · AS A PLATFORM

As a platform: the seam is the control plane, not the CLI

CampaignControlPlane — one protocol, adapters translate transport only

observe · events · steer · pause · resume · stop · artifacts · provenance · ingest_measurement · status · inflight · backtrack · request_human · submit_run · list_runs · run_result · await_run_result · cancel_run

in-process	HTTP + SSE	MCP
`control/plane.py::InProcessControlPlane`	`control/http.py::ControlPlaneServer`	`the same protocol, a third transport`

observe() returns what a policy sees

- The same CampaignObservation object a policy's decide() receives — informed monitoring means parity of evidence, not a progress bar.
- Admission is synchronous even over HTTP: 202 with a run id, or 409 with the gate and the reason that refused it.

Why admission cannot be deferred

- Accept-then-reject-later would sever a rejection from the request that caused it.
- And it cannot work anyway: the dry-run instantiates task agents, so admission has to happen where the toolkit is installed.

What is still missing — backlog C5, C6, C7

- RemoteSession and the routes it needs are covered end to end, but only tests/ ever constructs a server. There is no impress-a serve and no impress-a reason, so the split is reachable from the suite and not from a shell.
- Because of that, --model C builds an ExternalPolicy whose inbox can never be filled: it times out every 5 s and silently runs the model-D policy instead, while reporting itself as model C. It should refuse.
- And where the plane should live relative to the allocation is undecided — http.py is loopback, single trusted client, no auth, no TLS.

STATUS

What is real, and what is not — by dimension

FUNCTIONALITY	PERFORMANCE	USABILITY
Runs, for real	**Runs, for real**	**Runs, for real**
A six-stage real-toolkit campaign, end to end on Delta: 6/6 tasks, four objectives, a front.	The TRUSTED path: promoted after r0002, then 3 consecutive trusted cycles — no forced dry-run, no cost cap, no demotion.	A campaign is one YAML and one command; the whole local tier runs with no allocation.
The engine: compose, five gates, interlock, dry-run, Pareto tree, provenance, durable ledger.	Scrutiny measured and free: 2.8s of overhead per run, untrusted costs 0.0s [-0.9, 1.2].	Toolkits discovered at runtime: 5 of them, declarative specs plus a SKILL.md, validated at load.
Four control models plus conduct() reasoners over one executor — 11 tools, and adding one has never touched the composer.	Dragon startup AND teardown bounded; per-tool cost measured for all six real tools and the specs corrected from it.	Control plane in two transports, including a reasoner in another process driving a campaign over HTTP + SSE.
Has never run	**Has never run**	**Has never run**
replicas > 1 has NEVER executed on real tools, only against mocks. N independent lineages is the invariant most likely to be silently wrong.	No serial baseline and no head-to-head, so nothing in act 2 is a speedup.	No resume: the ledger and reattach() exist, nothing resumes from them.
No measurement has superseded a prediction, so calibration is untested — and that is the capability a composed graph has and a fixed one does not.	One GPU of four ever works: GPU-busy ≤17% as an upper bound, and packing has never been attempted.	impress-a serve / reason do not exist, so --model C fails open and reports a model-D campaign as model C.
6 of the 8 compute patterns have no member, P7 (an IMPRESS pipeline as one tool) among them.	No node has reached plain pass — all 5 trusted nodes missed an acceptance threshold, 31% over 16 runs. More draws, not code.	sites/*.yaml is read by no code. MCP is designed, not built. Nothing external has ever driven a campaign outside tests/.

Count the red column: it is the same length as the green one in all three. 16 complete six-stage runs and the trusted path held — every one of them a single lineage, on a system built for width.

DISCUSSION

Seven things I would like this room's opinion on

Grouped by what each one costs, not by who owns it — the numbering is the one the companion files use.

COSTS A WRONG ANSWER, OR REAL ALLOCATION TIME

1 rhapsody's monitor loop drops a whole sweep

performance

A worker exception that cannot be unpickled takes out every completion in that sweep, so a failing run reaches no terminal state. batch_task.get() is guarded; the follow-up get_stdout(block=False) re-reads the same entry and catches only OSError. Should it catch Exception and fail the ONE task?

2 Why does Dragon's Batch() stall?

performance

Bounded now, not diagnosed. Job 22318678 produced only infra-connect lines for two hours. Best guess: GPU-affinity worker rendezvous, or OFI negotiation under single-node mode. We fail fast enough to iterate — what should we instrument?

3 Can a running task's GPU ever be reclaimed?

performance

Future.cancel() returns False once started and asyncflow discards the answer, so we cannot even learn which happened. Is cooperative cancellation inside the task the only route, or is there a backend-level kill we should be using?

7 Is per-GPU packing reachable for us?

performance

Your own runs take GPU utilisation from 12% to 71% by putting 8 pipelines on a GPU. We are at ≤17% with three idle. Is that expressible in a rhapsody resource shape, or does it need Dragon placement we do not control?

COSTS WHOEVER PICKS THIS STACK UP NEXT

4 Nothing looks tasks up by workflow_id

usability

So "which tasks belong to run R" is answered by our own handle, which cancellation and draining both need. Is a lookup API wanted upstream, or is the tag meant to stay a label?

5 Does breadth belong in a pattern's identity?

functionality

Slide 18. Parameters are deliberately excluded from the signature; replica count is not. Is trust earned at one lineage evidence about four, when the cap it would lift exists to bound blast radius?

6 What is the cheapest real structural gate?

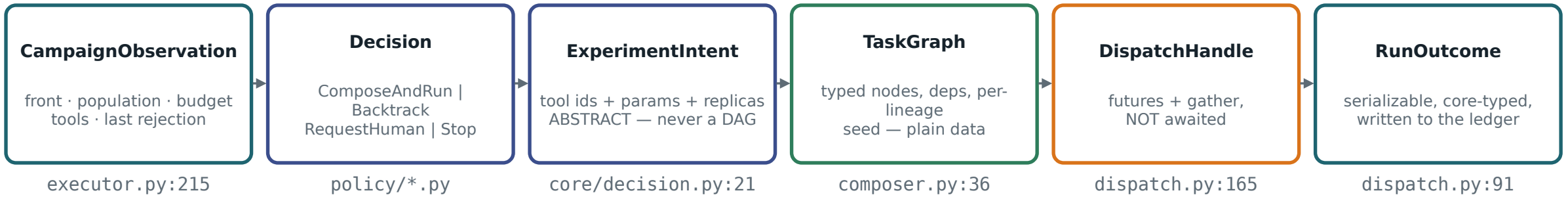
functionality

Every real gate we have is a threshold on a number the tool reports about itself. RFD3 gives us ligand_clashes in a sidecar we already open; you gate on it and we do not. For Rosetta you gate interaction_energy, which we never compute. Where would you start?

Every claim behind these is reproducible from this checkout, and every file:line on these slides is greppable from your laptop while I talk — including the five that point into your code rather than mine.

BACKUP · DATA FLOW

Backup — one experiment, and the type at every hop




```
$ impress-a run campaigns/mock-stabilize.yaml --model D

  campaign : mock-stabilize-D
  cycles   : 2
  stop     : target front size 3 reached
  nodes    : 4  front: 3
```

run.json — mined from a real run, this checkout

run	shape	nodes	est. gpu-h	admitted?
r0001	9fdcfd20…	12	0.165	**refused — interlock**
r0002	66bb4923…	8	0.110	yes
r0003	66bb4923…	8	0.110	yes

What the first row is

- provisional pattern capped at 10% of available cpu_hours (1.500 > 1.200)
- The policy answered by shrinking from 3 lineages to 2 and resubmitting. That is on_rejected doing its job: a refusal costs an attempt, not the experiment.

And what the rest of the run shows

- 4 design nodes over 2 cycles, every one marked suspect — the pattern stayed provisional: the front target was met after two clean runs, before the third that would promote it.
- Front of 3; the campaign stopped on "target front size 3 reached" rather than on budget.

The whole slide is one laptop run against the mock toolkit: real manager, real policies, real composer and validator, stubbed science.

Backup — eight compute patterns, and what each one forbids

core/types.py:12 — organised by scheduling implication, never by science

	What it is	What it forces on the orchestrator	Tools here
P1	GPU-node-local, in-job	must declare a GPU — a load-time error if not	7
P2	CPU-parallel fan-out, in-job	cores, not ranks; no topology to reserve	4
P3	MPI multi-node, in-job	a reserved rank layout; no resize mid-run	–
P4	external HPC job	durable job state — it outlives the agent	–
P5	network service over HTTPS	no local cost; quota, latency and outage	–
P6	in-process library call	never a scheduled node — inlined, or refused	–
P7	composite pipeline — an IMPRESS run	parameterize and observe; stages not steered	–
P8	external experiment (lab)	completes out of band, days later	–

```
class Pattern(str, Enum):      # 8 members, and only 2 rules derived from them
    def is_inline(self):   return self is Pattern.P6
    def is_external(self): return self in (Pattern.P4, Pattern.P8)

stages = [t for t in intent.stages           # composer.py - P6 never a node
          if self.reg.get(t).pattern is not Pattern.P6]
if spec.pattern is Pattern.P6:               # validate.py - gate 4 refuses one
    return ValidationFailure(gate="resource", node=tid, reason=...)
if self.pattern is Pattern.P1 and self.resources.gpus == 0:  # spec.py - at LOAD
    raise ValueError(f"{self.id}: P1 declared but resources.gpus == 0")
```

composer.py:45 · validate.py:106 · spec.py:107 · EDITED — @property elided

Organised by what it forces

- Not by what the tool computes. A tool is P4 for where it must be submitted, not because it runs MD.
- Which is why one generic factory serves every node: the differences that would have needed branching were settled before dispatch.

Two of the eight are in use

- All 11 registered tools are P1 (7) or P2 (4). P3–P8 have no member.
- So the P6-inline path, the P4 ledger and the P5 service path are contracts the suite asserts and no tool has taken.
- P7 is the one reserved for a reference IMPRESS pipeline run as a single tool of ours. It has no member either — so "IMPRESS becomes one of our tools" is a contract, not a

Dispatch does not branch on it

- The docstring says "dispatch depends on these." It does not, yet: exec/dispatch.py builds one closure per node, and exec/resources.py translates ResourceShape.
- So every consequence is enforced at load time or by a gate — 5 sites in 3 files, none under exec/ or runtime/.

Two senses of one word: a compute pattern is per tool, and is this slide; a composition pattern is a graph's shape (compose/graph.py:51) — that is what the trust ledger scores.

BACKUP · CONTROL MODELS

Backup — four control models over one executor

	model	mode	how decide() works	file
A	Four-node agentic loop	autonomous	A LangGraph graph: hypothesize → parameterize, auditable node by node	`policy/agentic.py`
B	Heavyweight LLM oracle	autonomous	One structured call over rich campaign state	`policy/oracle.py`
C	External caller steering	headless	Blocks on the control plane for a directive	`policy/external.py`
D	Explicit policy	autonomous	Rules, bandit, Bayesian optimization, or replay — fully reproducible	`policy/explicit.py`

They compose rather than inherit

- RuleCorrectionsPolicy wraps any policy in an unconditional external guard; LoggingPolicy wraps any policy for provenance. Decorators, not subclasses.
- Writing a fifth is subclassing BasePolicy and implementing one method.

The claim the whole design rests on

- Everything below the policy layer is byte-identical across A, B, C and D. A campaign run by a hand-written rule cascade exercises the same machinery as one steered by a frontier LLM.

One editorial point, since Flowgentic's authors are here

- The outer loop is still ours, deliberately — it is just two coroutines now instead of one. Ceding it to a framework pushes the validation gates, the dry-run and the interlock into awkward places.
- Flowgentic's own README reaches the same conclusion: it recommends the augmented rung — the application owns the loop, agents are bound at decision points — over the fully agentic one.